

Tech Fest Registration — Backend Hiring Challenge

IEEE Student Branch RVCE · Backend Development Track

Overview

The club is launching registrations for its biggest tech fest of the year. Your task is to build the central backend API that powers the event — from a student signing up, to payment confirmation, to a volunteer scanning their QR code at the gate.

This is a real system with real constraints. How you design, secure, and protect it is the challenge.

What to Build

A **REST API** that serves a tech fest registration system with two user roles:

Role	What they can do
Student	Register, log in, view their own ticket
Volunteer	View all registrations, check students in at the gate

Mandatory Requirements

The following must be implemented. Everything else — structure, additional endpoints, tooling choices — is up to you. Any additional functionality based on your innovative ideas will be considered for evaluation.

1. Authentication
 2. Registration
 3. Tickets
 4. Check-in
 5. Payment
-

Technical Constraints

- **Language / framework:** Your choice. Document it.
- **Database:** Your choice. SQLite is perfectly fine.
- **No frontend required.** API responses only.

Submission

1. Push your code to a **public GitHub repository**.
2. Include a `README.md` with:
 - How to run the project locally (exact commands)
 - How to create a volunteer account for testing
 - Any assumptions you made
 - **Documentation of all API endpoints** (path, HTTP methods, request parameters/body, and response format)
3. Provide a **Postman shared URL** or include the **Postman collection JSON file** in your repository to test the endpoints.
4. Include a `SCALE.md` answering the following:

Registrations open on Friday at 6:00 PM. We expect 2,000 students to hit the registration endpoint within the first 60 seconds. The server has 1 GB of RAM. What is one concrete strategy — implemented or explained in detail — that you would use to keep the server stable during this spike?

1. Submit the repository link via [Google Form](#).
-

Evaluation

Your submission will be evaluated on:

- Whether the mandatory requirements work correctly under normal and edge-case conditions
- How your system behaves when things go wrong — duplicate requests, invalid input, unauthorised access
- Code organisation and clarity
- The depth and practicality of your `SCALE.md` answer
- Any additional functionality or innovative features you build to enhance the application

The requirements described above contain deliberate edge cases that reflect the realities of production systems. How your API handles them — crashing, behaving incorrectly, or responding gracefully — is a significant part of what we are evaluating.

Good luck.
